



AssetFlow

Smart Asset Management and Resource
Allocation Platform

Design Document

Technology Stack

React.js · Node.js · Express.js · MongoDB Atlas
· JWT · Recharts · Vercel · Render

Team Members:

Ch.Rajeev Lochan | A.Iswarya | M.Sudhishna

1. Problem Understanding

The Cultural Council of IIT Roorkee manages a large pool of shared resources — DSLR cameras, studio lighting, audio systems, costumes, stage props, and recording equipment — across multiple sections and events. Current coordination relies on manual registers, spreadsheets, and informal communication channels, resulting in:

- No centralized visibility of asset availability
- Scheduling conflicts due to double-bookings
- No accountability or audit trail for issued assets
- Delayed returns with no due-date enforcement
- Zero analytics for utilization planning

The goal is to build a centralized, web-based asset management platform that handles inventory tracking, booking requests, approval workflows, physical issuance, return management, and operational analytics — while remaining simple and maintainable.

2. System Architecture

2.1 Architectural Overview

The system follows a three-tier architecture with clear separation between the presentation layer (React SPA), the application layer (Express REST API), and the data layer (MongoDB Atlas).

Layer	Technology	Responsibility
Frontend	React.js, Vite, React Router, Recharts	SPA — UI, routing, charts, role-based views
Backend API	Node.js, Express.js	REST endpoints, business logic, JWT auth
Database	MongoDB Atlas, Mongoose ODM	Data persistence, schema validation, querying
Auth	JWT, bcryptjs	Token generation, password hashing, stateless sessions
Deployment	Vercel (frontend), Render (backend)	Cloud hosting, auto-deploy from GitHub

2.2 Component Interaction

The system uses a standard three-tier request flow. The React SPA sends authenticated HTTP requests (JWT in the Authorization header) to the Express backend on Render, which validates the token, enforces role permissions, and queries MongoDB Atlas via Mongoose ODM. All API routes are prefixed `/api/` and grouped by resource: auth, assets, bookings, allocations, analytics, audit-logs, and users.

2.3 Role-Based Access

Two roles are supported — admin and user. Role enforcement happens at both the frontend (route guards via `ProtectedRoute`) and the backend (`role()` middleware on sensitive routes).

Page	Admin	User
Dashboard	✓	✓
My Bookings	✗	✓
Approvals / Allocations / Users / Analytics / Audit Logs	✓	✗
My Analytics	✗	✓
Add/Edit/Delete Asset	✓	✗

3. Database Schema

Schema managed via Mongoose ODM on MongoDB Atlas. Core collections described below.

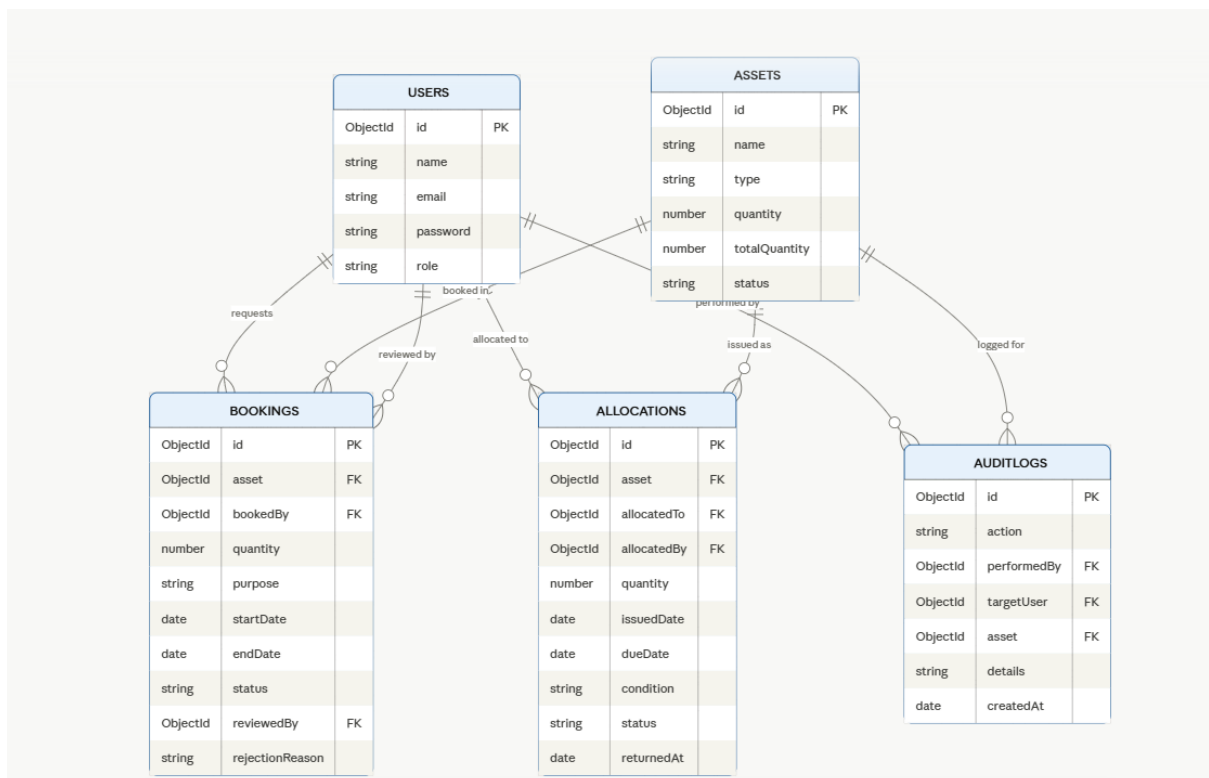
3.1 Core Collections:

Collection	Key Fields	Notes
users	name, email, password, role	role: enum [user, admin]. Password bcrypt-hashed
assets	name, type, quantity, totalQuantity, status	quantity tracks current available; totalQuantity stores original for utilization calculation
bookings	asset, bookedBy, quantity, purpose, startDate, endDate, status, reviewedBy, rejectionReason	status: enum [pending, approved, rejected]. Links user to asset request
allocations	asset, allocatedTo, allocatedBy, quantity, issuedDate, dueDate, condition, status, returnedAt	status: enum [active, returned]. Created automatically on booking approval
auditlogs	action, performedBy, targetUser, asset, details, createdAt	Tracks all critical admin actions with full context

3.2 Key Design Decisions

- **Dual quantity fields:** quantity tracks currently available stock while totalQuantity stores the original amount, enabling accurate utilization rate calculation without additional queries.
- **Booking → Allocation pipeline:** Bookings represent requests. Allocations represent physical issuance. Separating them keeps the approval workflow independent from return tracking.
- **Automatic audit logging:** All approve, reject, allocate, and return actions write to AuditLog automatically at the service layer — no manual logging required.
- **Timestamps via Mongoose:** { timestamps: true } on Booking and Allocation schemas auto-populates createdAt and updatedAt for trend analytics.

4. Entity Relationship Diagram



6. API Overview

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register new user
POST	/api/auth/login	Public	Login, returns JWT
GET	/api/assets	Auth	Get all assets
POST	/api/assets/add	Admin	Add new asset
PUT	/api/assets/:id	Admin	Edit asset
DELETE	/api/assets/:id	Admin	Delete asset
POST	/api/bookings	Auth	Submit booking request
GET	/api/bookings/my	Auth	Get my bookings
GET	/api/bookings/pending	Admin	Get pending requests
GET	/api/bookings/all	Admin	Get all requests
PUT	/api/bookings/:id/approve	Admin	Approve booking, creates allocation
PUT	/api/bookings/:id/reject	Admin	Reject with optional reason
POST	/api/allocations	Admin	Manually allocate asset to user
GET	/api/allocations	Admin	Get active allocations
PUT	/api/allocations/:id/return	Admin	Mark returned, restore quantity
GET	/api/users	Admin	Get all users
DELETE	/api/users/:id	Admin	Delete user
GET	/api/analytics/summary	Admin	Summary stats
GET	/api/analytics/bookings-per-asset	Admin	Bar chart data
GET	/api/analytics/bookings-trend	Admin	7-day line chart data
GET	/api/audit-logs	Admin	All audit logs (last 200)

6. Design Decisions

6.1 JWT Stateless Authentication

Stateless JWT tokens were preferred over session-based auth to eliminate server-side session storage and allow the frontend (Vercel) and backend (Render) to operate as fully independent services. Tokens expire in 7 days and are verified on every request via Express middleware.

6.2 Booking and Allocation as Separate Entities

Bookings represent user-initiated requests pending admin review. Allocations represent confirmed physical issuance created automatically when a booking is approved. This separation allows return tracking, condition logging, and overdue detection without polluting the booking workflow.

6.3 Quantity-Based Inventory Model

Rather than treating each asset unit individually, assets store aggregate quantity (available) and totalQuantity (original). This simplifies queries and UI while supporting partial bookings — an asset is only marked booked when quantity reaches zero.

6.4 Automatic Audit Logging

All critical admin actions — approve, reject, allocate, return — write an AuditLog entry at the route handler level. This provides a full accountability trail without requiring separate admin input, and supports the Audit Logs page with filterable action types.

6.5 Role Enforcement at API Layer

A reusable role(["admin"]) middleware wraps all admin-only routes on the backend. Frontend route guards (ProtectedRoute) provide UX-level protection, but the API remains secure regardless of frontend behavior.

6.6 Recharts for Analytics

Recharts was chosen for its native React integration and lightweight footprint. The analytics dashboard renders a line chart (7-day booking trend), bar chart (bookings per asset), and pie chart (available vs. booked) — all from dedicated backend endpoints that pre-aggregate data

7. Feature Completeness Summary

Feature	Status
User Authentication (JWT, register/login)	Done
Role-based access (admin / user)	Done
Inventory management (add/edit/delete/categorize)	Done
Asset search and type filter	Done
Booking requests (quantity, purpose, dates)	Done
Approval workflow (approve/reject with reason)	Done
Asset issuance and return tracking	Done
Overdue allocation detection	Done
Analytics dashboard (bar, line, pie charts)	Done
User booking history and personal analytics	Done
Admin user management	Done
Audit logs with action filtering	Done
Deployed (Vercel + Render + MongoDB Atlas)	Done